Computational Communication Science 2
Week 2 – Lecture
Text Representation: Preprocessing, Vectorizers, and Embeddings

Anne Kroon
a.c.kroon@uva.nl

April 7, 2026

Digital Society Minor, University of Amsterdam

Today's Agenda

Recap

Preprocessing in depth

- Tokenization
- Stemming and lemmatization

Building text representations: vectorizers

- From text to vectors
- Weighting: TF-IDF
- Pruning

Introducing embeddings

- Word embeddings

Recap

Review of Key Concepts

- **Text as data:** digital communication produces large amounts of text
- **Two approaches:**
  - *Bottom-up*: explore patterns without predefined categories
  - *Top-down*: measure predefined concepts
- **Preprocessing:** cleaning and preparing raw text
  - Tokenization, stopword removal, basic string methods

Today we go deeper into preprocessing and move on to text representation.

Preprocessing in depth

Preprocessing in depth

Tokenization

Tokenization: a reminder

Goal: split a string into a list of tokens.

```
from nltk.tokenize import TreebankWordTokenizer

docs = ["Communication in the digital society is very complex",
        "I like to study it"]

tokens = [TreebankWordTokenizer().tokenize(d) for d in docs]
print(tokens)
```

```
[['Communication', 'in', 'the', 'digital', 'society',
  'is', 'very', 'complex'],
 ['I', 'like', 'to', 'study', 'it']]
```

Result: one list of tokens per document.

What is a token?

A **token** is the basic unit of analysis after splitting text.

Tokens are usually single words, but can also be sequences of words:

Type	n	Example
Unigram	1	`"digital"`, `"society"`
Bigram	2	`"digital society"`
Trigram	3	`"the digital society"`

Why multi-word tokens? Single words sometimes lose important context.
"not good" means something very different from *"good"*.

In scikit-learn, set `ngram_range=(1,2)` in `CountVectorizer` to include both unigrams and bigrams.

Preprocessing in depth

Stemming and lemmatization

Stemming vs. lemmatization

Both reduce token forms to a common base — but differently.

Stemming

- Chops off endings using rules
- Fast, but can produce non-words
- `"society"` → `"societi"`

Lemmatization

- Maps to dictionary form
- Slower, but linguistically accurate
- `"better"` → `"good"`

Stemming in practice

Note: stemming is applied *per token* — loop over documents *and* tokens.

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

# tokens is a list of lists
tokens_stemmed = [[stemmer.stem(token) for token in doc]
                  for doc in tokens]
print(tokens_stemmed)
```

```
[['commun', 'in', 'the', 'digit', 'societi', 'is', 'veri', 'complex'],
 ['i', 'like', 'to', 'studi', 'it']]
```

Stemming can produce non-standard forms (`societi`, `veri`) — this is expected.

Stemming is a heuristic, not a dictionary lookup.

Lemmatization in practice: NLTK

Like stemming, lemmatization is applied *per token*.

With NLTK's `WordNetLemmatizer`, the default assumes every token is a **noun**:

```
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()

words = ["studies", "geese", "running", "better"]
print([lemmatizer.lemmatize(w) for w in words])
```

```
['study', 'goose', 'running', 'better']
```

Problem: `"running"` and `"better"` are not reduced — the lemmatizer does not know they are a verb and adjective. Fixing this requires providing POS tags manually, which is impractical at scale.

Lemmatization in practice: spaCy

spaCy handles POS tagging *automatically* — no manual tags needed.

```
import spacy
nlp = spacy.load("en_core_web_sm")

doc = nlp("The studies show that running is better than doing nothing")
# show only tokens where lemma differs from the original
print([(t.text, t.lemma_) for t in doc if t.text != t.lemma_])
```

```
[('studies', 'study'), ('running', 'run'),
 ('is', 'be'), ('better', 'good'), ('doing', 'do')]
```

`"running"` → `"run"`, `"better"` → `"good"`, `"studies"` → `"study"` — all correct, automatically.

Building text representations: vectorizers

Building text representations: vectorizers

From text to vectors

A text as a bag of words

The simplest representation: count how often each token appears.

```
from collections import Counter

t = "This this is is is a test test test"
print(Counter(t.split()))
```

```
Counter({'is': 3, 'test': 3, 'This': 1, 'this': 1, 'a': 1})
```

This representation:

- preserves token frequencies
- does *not* preserve word order
- can be treated as a **vector** we can calculate with

Note: `This` and `this` are counted separately — lowercasing matters!

From vectors to a matrix

Stacking one vector per document gives a **document-term matrix (DTM)**.

```
t1 = "This this is is is a test test test"
t2 = "This is an example"
```

	a	an	example	is	This	this	test
t1	1	0	0	3	1	1	3
t2	0	1	1	1	1	0	0

Each row = document; each column = token; each cell = count.

Note: `This` and `this` are separate columns — treated as different tokens without lowercasing!

What can you do with such a matrix? Why represent texts this way?

What is a vectorizer?

- Automates building a DTM from raw text
- First **fitted** on a corpus: learns which **tokens** exist and assigns them to columns
- Can then **transform** new texts into the same feature space
- Result: a **sparse matrix** — only non-zero values stored

CountVectorizer: basic usage

```python
from sklearn.feature_extraction.text import CountVectorizer

docs = ["Communication in the digital society is very complex",
        "I like to study it"]

cv = CountVectorizer()
dtm = cv.fit_transform(docs)

print(cv.get_feature_names_out())
print(dtm.toarray())
```

```
['communication' 'complex' 'digital' 'in' 'is' 'it'
 'like' 'society' 'study' 'the' 'to' 'very']

[[1 1 1 1 1 0 0 1 0 1 0 1]
 [0 0 0 0 0 1 1 0 1 0 1 0]]
```

Note: CountVectorizer lowercases and removes punctuation by default.

Building text representations: vectorizers

Weighting: TF-IDF

Are all tokens equally informative?

Raw counts treat all tokens the same.

- `"the"` appears in almost every document — tells us very little
- `"misinformation"` in only a few documents — much more distinctive

Idea: downweight frequent tokens, upweight rare ones.

$\Rightarrow$ This is what **TF-IDF** does.

TF-IDF: the formula

$$\text{tf-idf}_{i,j} = \text{tf}_{i,j} \times \log\left(\frac{N}{\text{df}_i}\right)$$

- $\text{tf}_{i,j}$ = how often token i appears in document j
- df_i = number of documents containing token i
- N = total number of documents

Intuition:

- High TF + low DF $\rightarrow$ high score (important for this doc, rare overall)
- High DF $\rightarrow$ low score (common tokens like `"the"`)

scikit-learn applies additional normalization — see documentation.

TF-IDF vs. raw counts

Raw counts

- Interpretable
- Describes a document on its own
- Good when frequency itself matters

TF-IDF

- Adjusts for common tokens
- Better for comparing documents
- Good for search and classification

Bottom line: it is an empirical question — try both and evaluate.

Building text representations: vectorizers

Pruning

Pruning: reducing the feature space

- **Stopword removal:** fixed list, does not adapt to your corpus
- **TF-IDF:** downweights frequent tokens, but keeps them in the matrix
- **Pruning:** removes tokens based on empirical frequency in *your* corpus

Parameters in scikit-learn:

- `max_df`: drop tokens appearing in more than X% of documents
- `min_df`: drop tokens appearing in fewer than *n* documents

Putting it all together

Step 1: stopwords only

```
cv = CountVectorizer(stop_words=mystopwords)
```

Step 2: add a better tokenizer

```
cv = CountVectorizer(
    tokenizer=TreebankWordTokenizer().tokenize,
    stop_words=mystopwords)
```

Step 3: add pruning

```
cv = CountVectorizer(
    tokenizer=TreebankWordTokenizer().tokenize,
    stop_words=mystopwords,
    max_df=0.75, min_df=2)
```

Step 4: switch to TF-IDF

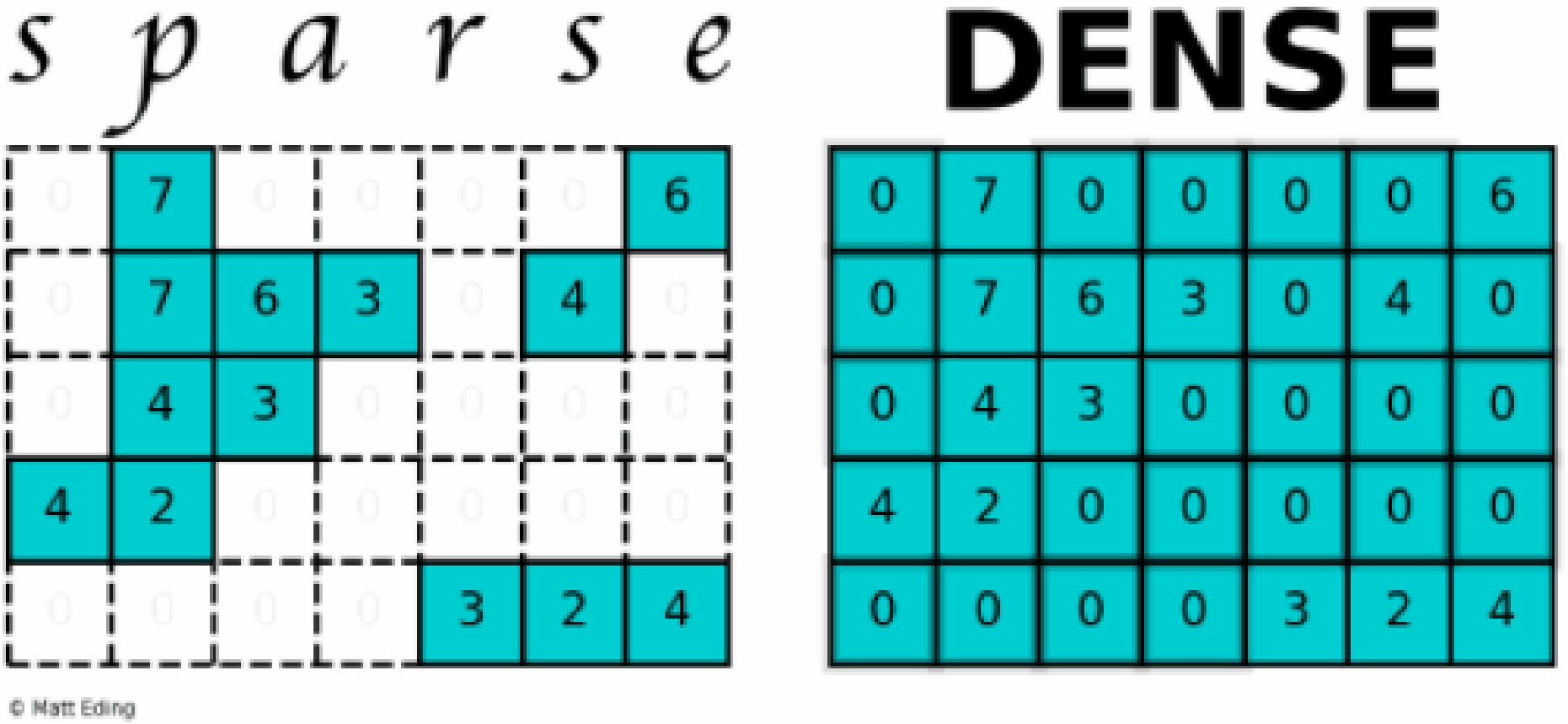
```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(
    tokenizer=TreebankWordTokenizer().tokenize,
    stop_words=mystopwords,
    max_df=0.75, min_df=2)
```

Which combination is "best" — and how do you decide?

Sparse matrices

Why sparse?

- A DTM can have tens of thousands of columns (one per token)
- Most cells are zero — storing all of them wastes memory
- **Sparse matrix:** only non-zero values and positions are stored
- Never convert to dense / DataFrame on real data!



https://matteding.github.io/2019/04/25/sparse-matrices/

Explore vectorizers interactively

Best of both worlds

Use `CountVectorizer` or `TfidfVectorizer` with an NLTK-based tokenizer for full control.

Try it out yourself: DTM Explorer (Streamlit app)

Go further in code

try it out

Use `CountVectorizer` or `TfidfVectorizer` with an NLTK-based tokenizer.

Notebook: Understanding vectorizers

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

Introducing embeddings

Why should communication scientists care about embeddings?

As communication researchers, we often want to know:

- Are these two news articles about the same topic?
- How similar is the language used by different politicians?
- Does the meaning of a word (e.g., *"populism"*) shift over time?

TF-IDF cannot answer these questions well — it treats every token as independent and has no notion of meaning.

Word embeddings open the door to meaning-aware text analysis.

The problem with bag-of-words

Consider these two sentences:

- *"The president gave a speech about climate change."*
- *"The leader delivered a talk on global warming."*

To a human, these sentences mean almost the same thing.
To a bag-of-words model, they share **only one token**: `"a"`.

Why? Because `"president"` and `"leader"`, `"speech"` and `"talk"`, `"climate change"` and `"global warming"` are treated as completely unrelated.

We need a representation that captures **meaning**, not just token identity.

From sparse to dense: the key idea

Bag-of-words (sparse): one dimension per token in the vocabulary.

`king = [1, 0, 0, 0, 0, 0, ..., 0]` (tens of thousands of dimensions)

Word embedding (dense): a short vector of real numbers.

`king = [0.32, −0.14, 0.87, 0.05, −0.61, ...]` (100–300 dimensions)

Each number captures some **latent aspect of meaning** — we do not define what each dimension means; the model figures it out from data.

Words used in similar contexts end up with **similar vectors**.

Introducing embeddings

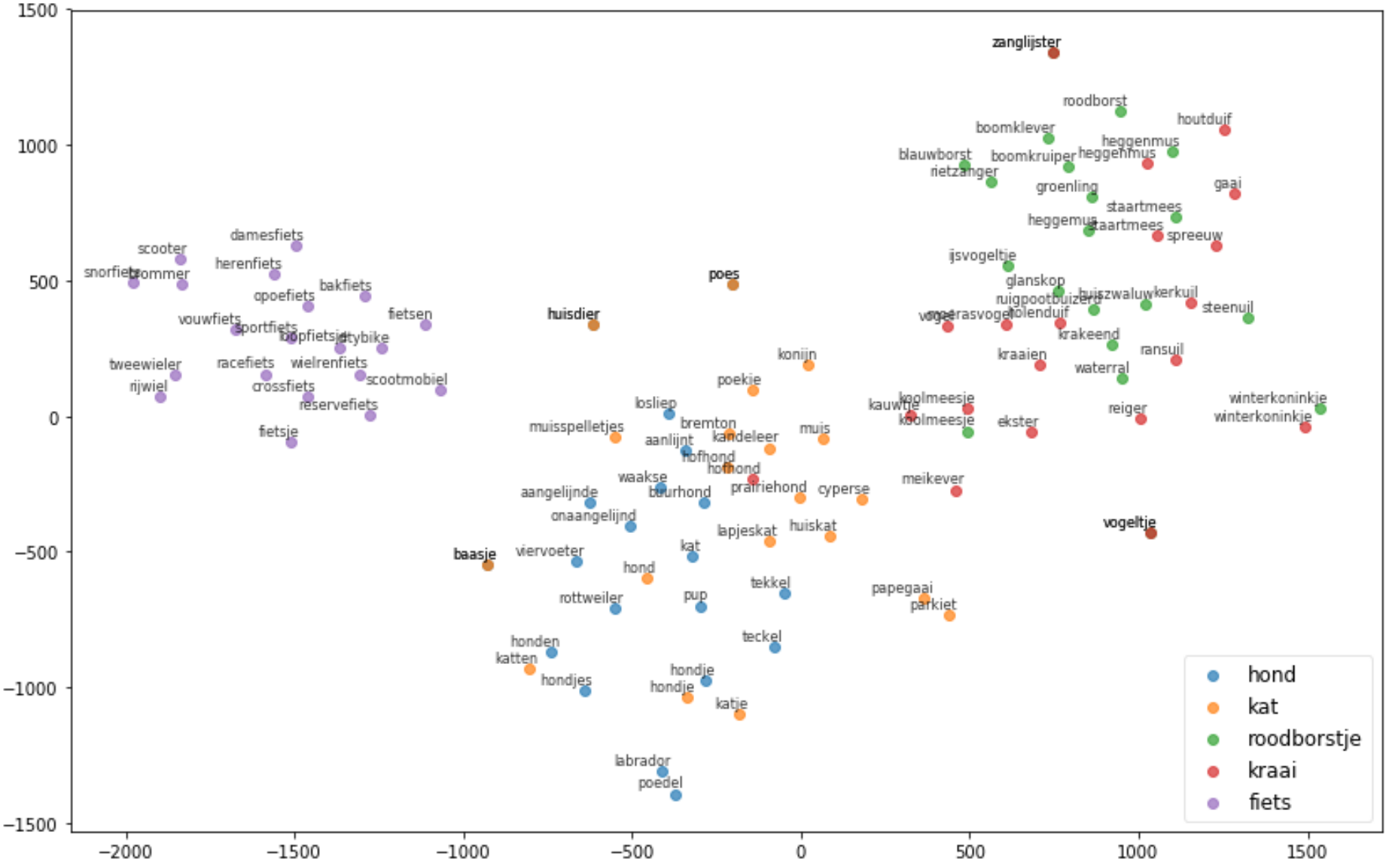
Word embeddings

How are embeddings learned?

Core idea (Firth, 1957)

"A word is characterized by the company it keeps."

- A model is trained on a large corpus (e.g., Wikipedia, news articles)
- It learns to **predict** which words appear near each other
- Words that appear in similar contexts get **similar vectors**
- No manual labelling needed — the signal comes from the text itself
- Popular models: **Word2Vec**, **GloVe**, **fastText**


zanglijster
roodborst
houtduif
boomklever
heggenmus
blauwborst
boomkruiper
heggenmus
rietzanger
groenling
gaai
staartmees
heggemus
staartmees
spreeuw
damesfiets
scooter
herenfiets
snorfiets
bommer
bakfiets
opoefiets
fietsen
vouwfiets
sportfiets
hoopfiets
ditybike
tweewieler
racefiets
wielrenfiets
rijwiel
crossfiets
scootmobiel
reservefiets
fietsje
poes
huisdier
ijsvogeltje
glanskop
ruigpootbuizerd
huiszwaluw
kerkuil
steenuil
vogelerasvogel
holenduif
krakeend
kraaien
ransuil
waterral
koniijn
poekie
losliep
bremton
muisspelletjes
aanlijnt
kandeleer
muis
hofhond
hofhond
waakse
prairiehond
cyperse
aangelijnde
buurhond
onaangelijnd
lapjeskat
huiskat
kat
baasje
viervoeter
hond
tekkel
rottweiler
pup
teckel
honden
katten
hondje
hondje
hondjes
katje
labrador
poedel
kauwtje
koolmeesje
koolmeesje
ekster
reiger
winterkoninkje
winterkoninkje
meikever
vogeltje
papegaai
parkiet
hond
kat
roodborstje
kraai
fiets

Similar words, similar vectors

Because embeddings are learned from context, words with related meanings end up **close together** in vector space.

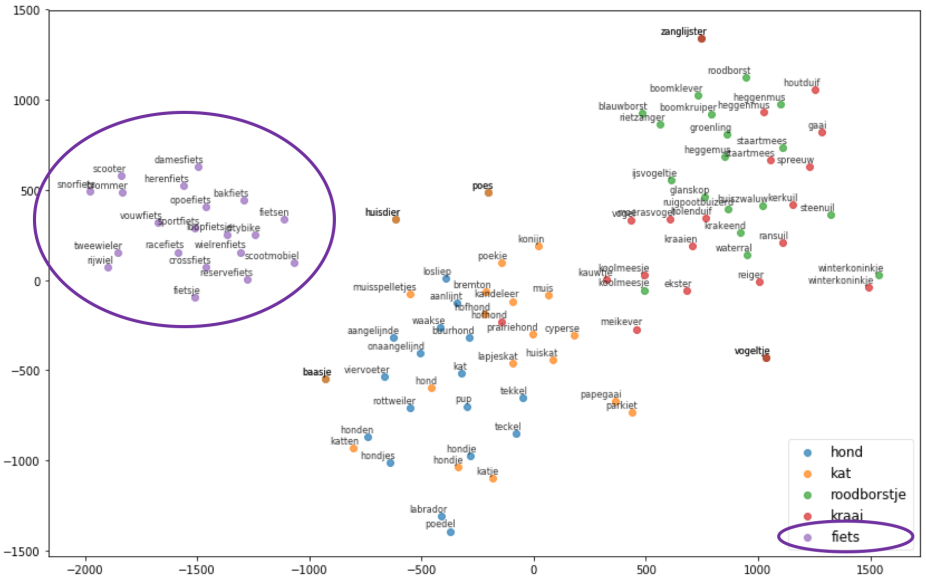
- `"dog"` and `"cat"` are close — both are pets
- `"king"` and `"queen"` are close — both are royalty
- `"happy"` and `"joyful"` are close — synonyms

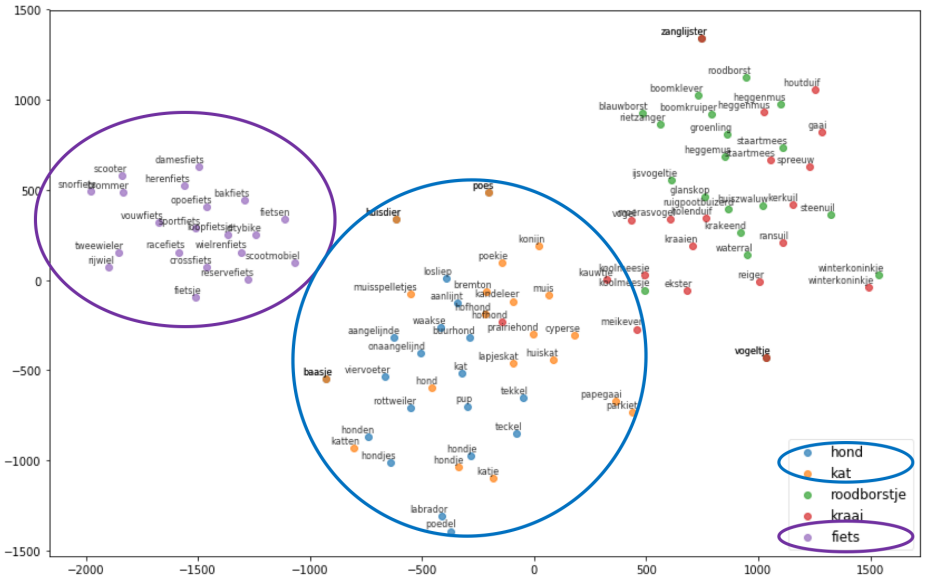
This also means you can do **arithmetic** with meaning:

$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$$

The geometry of the vector space reflects real-world relationships — something TF-IDF vectors cannot do.

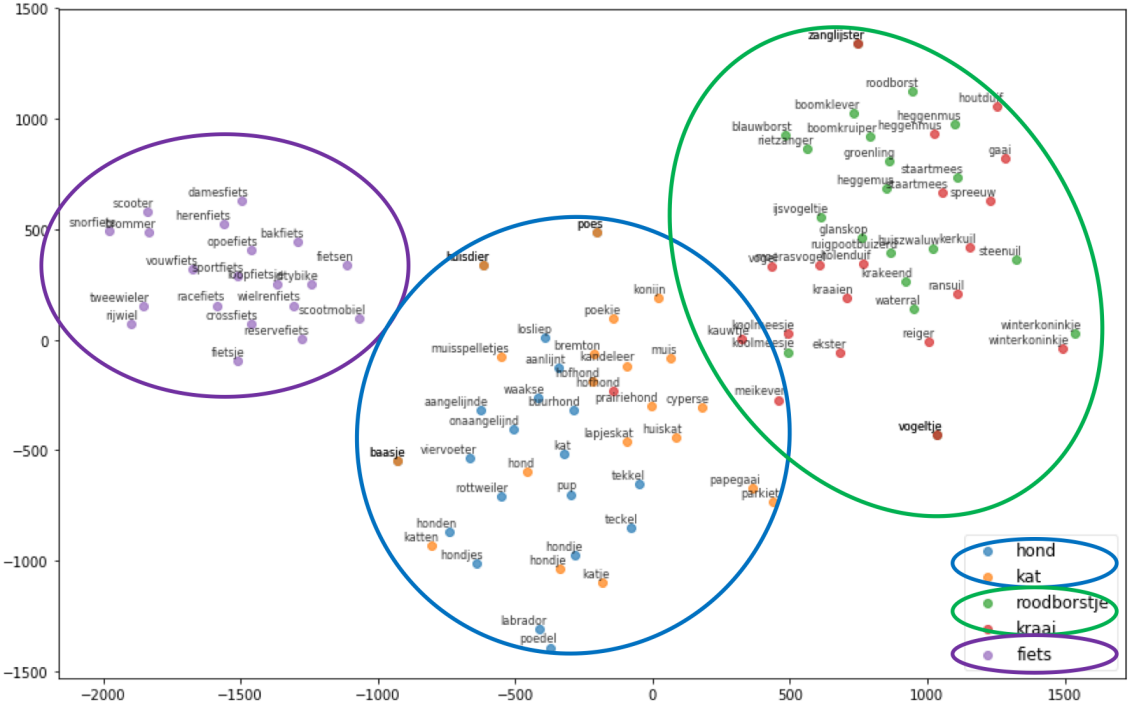
Visualizing word embeddings





Words with similar meanings cluster together in vector space.

Explore embeddings interactively



- TensorFlow Projector — explore word embeddings in 3D

Why use pretrained embeddings?

Training embeddings from scratch requires **massive amounts of text** and computing power.

Pretrained models are trained once on large corpora and shared openly:

- **Word2Vec** (Google News)
- **GloVe** (Common Crawl)
- **fastText** (subword info)
- **BERT** (contextualized)

Note: Word2Vec and GloVe give each word *one fixed vector*. BERT gives a word a *different vector depending on its context* — `"bank"` in *"river bank"* vs. *"bank account"* gets different vectors. We will return to this later.

Embeddings in communication science

What can we do with embeddings that we could not do before?

- **Sentiment analysis:** understand audience reactions beyond keyword matching
- **Recommender systems:** suggest content based on semantic similarity
- **Text similarity:** measure how alike two texts are in meaning

Note: we will work with embeddings hands-on next week.

Embeddings in practice: spaCy

Notebook: If you want to visualize your own embeddings

Wrap-up

Key takeaways

- **Preprocessing** shapes what your model sees — choices matter
- **Vectorizers** turn text into numerical matrices of token counts
- **TF-IDF** improves on raw counts by downweighting common tokens
- **Pruning** removes very rare or very frequent tokens
- **Bag-of-words ignores meaning** — synonyms and related words are unrelated features
- **Word embeddings** capture meaning and context by learning from large corpora
- There is no single best approach — it depends on your data and question

Next week

- Text similarity and distance measures
- Building a simple content-based recommender system
- Using embeddings for document comparison

Thank you!

Questions or comments?

- | a.c.kroon@uva.nl

References i